6. RESULTS AND DISCUSSION

6.1 Test Reports

Module 6 was tested after completing the analytics and interview-history features. The objective was to confirm that a logged-in user can view only their completed interviews, inspect saved performance reports, filter history records and see graph-ready analytics generated from MongoDB data. The tests were performed using the React frontend, Express API, JWT authentication middleware and MongoDB Atlas.

Test Condition	Input / Action	Expected Result	Actual Result
History API authorization	Call GET /api/interviews/history with JWT	Only logged-in user's completed interviews are returned	Pass - success:true response received
Analytics API	Call GET /api/interviews/analytics with JWT	Summary, category averages, trend and suggestions returned	Pass - safe empty response returned when no interview exists
Empty analytics	New user with zero completed interviews	Dashboard shows friendly empty state; no crash	Pass
Completed interview	Finish every question and save final report	Interview appears in history with score and date	Pass
History filter	Search role; choose type and difficulty	Only matching completed interviews displayed	Pass
Previous report	Select View from history row	Stored report opens at /interview/:id/report	Pass
Pagination	More than ten completed interviews	Previous/Next controls load correct page	Pass
Responsive interface	Open analytics on smaller screen	Cards, filters and history remain readable	Pass

6.1.1 Results Discussion

The protected API checks demonstrated that analytics are user-specific. The returned history object contains completed interviews only, while the analytics object contains total interviews, average score, highest score, latest score, improvement value, category averages, score-trend points and repeated suggestions. When the account has no completed interviews, null scores and empty arrays are intentionally returned; this prevents invalid calculations and gives a clean first-use experience.

After interviews are completed, the dashboard uses the stored performanceReport from Module 5 instead of recalculating every answer. This makes the history screen faster and supports direct navigation to previous reports. The score-trend component uses SVG and does not add an external chart dependency, keeping the frontend lightweight.

6.1.2 Module 6 Components

Component	Purpose	Result

Component	Purpose	Result
GET /api/interviews/history	Returns paginated, filterable history	Protected and user-specific
GET /api/interviews/analytics	Calculates overall progress data	Graph-ready JSON response
AnalyticsDashboard.jsx	Loads analytics and history APIs	Protected analytics page
ProgressSummary.jsx	Displays totals and score cards	Shows empty values safely
PerformanceChart.jsx	Displays score trend using SVG	No additional chart library
InterviewHistory.jsx	Provides filters, paging and report links	Previous reports accessible

6.2 User Documentation

This section explains how a user operates the performance analytics module after completing at least one mock interview. The Analytics page is protected, therefore the user must first register and log in.

6.2.1 Opening Performance Analytics

- Start the backend server and frontend development server, then open the platform in the browser.
- Log in using a registered account.
- Open the Dashboard and click the Performance analytics button.
- The system opens the protected route: /analytics.
- If no interview is completed, the page shows an empty-state message and a Start new interview button.

6.2.2 Completing an Interview to Create Analytics

- Generate an interview from the resume profile. Five questions are recommended for quick testing.
- Type an answer or select Answer using microphone, allow microphone access, speak, stop voice input and edit the transcript if needed.
- Select Save and continue for every question. The answer is evaluated and saved.
- On the final question, select Save and finish. The final performance report opens automatically.
- Return to the dashboard and open Performance analytics again to see the score and interview-history entry.

6.2.3 Understanding the Analytics Page

Area	What the user sees	How to use it
Summary cards	Completed interviews, average score, highest score and improvement	Compare current progress with earlier interviews
Score trend	One point per completed interview	Hover or read labels to compare scores over time
Category analysis	Correctness, confidence, communication and technical accuracy	Focus practice on the weakest category

Area	What the user sees	How to use it
Improvement plan	Repeated evaluation suggestions	Use frequent suggestions as a revision checklist
Interview history	Role, type, difficulty, score, date and View action	Review any saved report
Filters	Role search, interview type and difficulty	Narrow history results without affecting stored data

6.2.4 Demonstration Procedure

For a classroom demonstration, create and complete two short interviews with the same account. Open Performance analytics, show both score points in the graph, select a type or difficulty filter, and click View on an older interview. Refresh the browser page to demonstrate that the report and history come from MongoDB Atlas rather than temporary frontend state.

6.2.5 Screenshots of Module 6

The following screenshots show the implemented performance analytics interface before completed interviews are available. The empty-state design clearly guides a new user to complete an interview, while preserving the same summary, graph, category and filtering layout used for completed data.

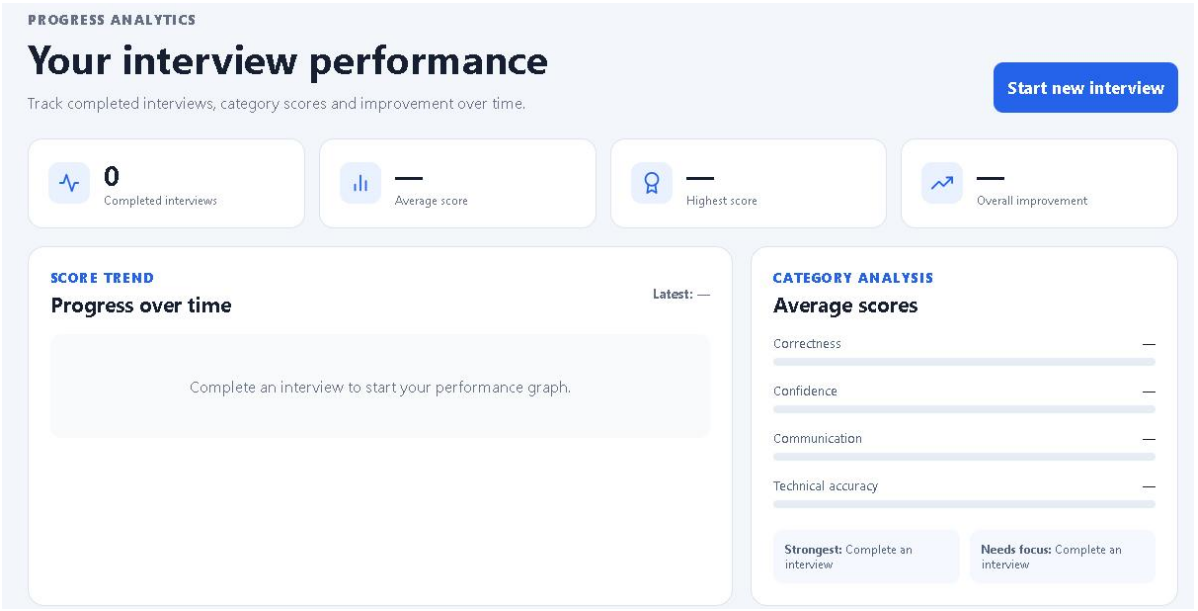


Figure 6.1: Performance Analytics overview - summary cards, score trend and category analysis.

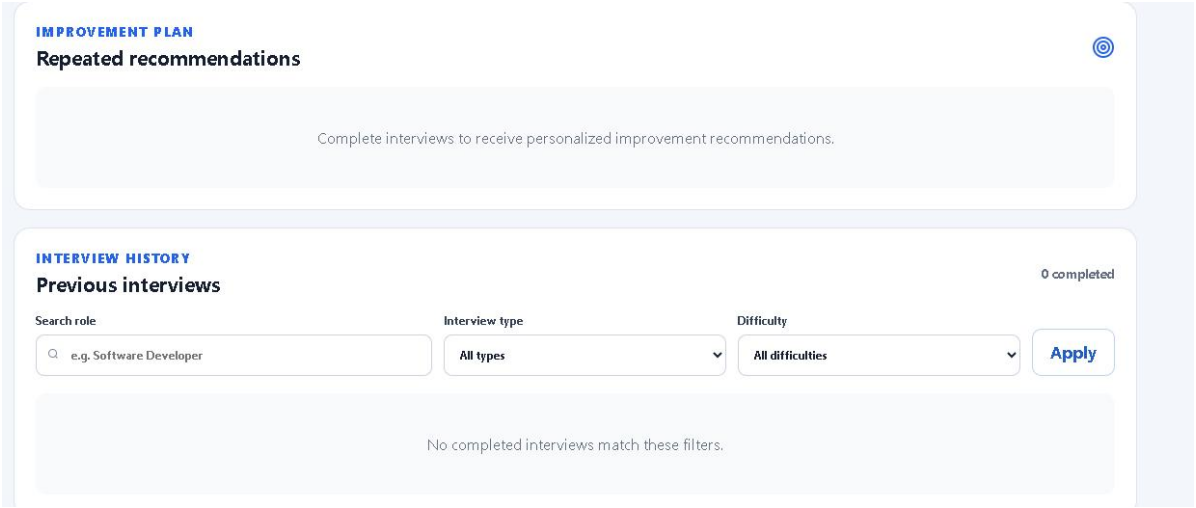


Figure 6.2: Improvement plan and Previous Interviews section with role, type and difficulty filters.

6.3 Conclusion

Module 6 completes the user-progress layer of the AI Interview Preparation Platform. The results confirm that completed interviews are stored permanently, shown only to their owner and transformed into meaningful scores, trends and recommendations. The module gives users a practical way to track improvement and prepares the application for final deployment and security improvements in Module 7.